

University
Politehnica
of Bucharest





PL/SQL (VI)

Ș.L. Dr. Ing. Ciprian-Octavian Truică
ciprian.truica@upb.ro



Overview

- Packages
- Triggers



Overview

- Packages
- Triggers



Packages

- A package is a schema object that groups logically related PL/SQL:
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions
- A package is compiled and stored in the database
- Applications can share the contents of a package
- A package contains two parts
  - A package specification (mandatory)
  - A package body (not mandatory)



Packages

- A package always has a specification where public items are declared
- Public items can be referenced from outside the package
- The package must also have a body if the public items include
  - Cursors (the cursors' queries should be defined in the body)
  - Subprograms (the code for public subprograms are defined in the body)
- The body can also declare and define private items that cannot be referenced from outside the package
- The body can have
  - An initialization part containing statements to initialize variables and do other one-time setup steps
  - An exception-handling part
- The body can be changed without changing
  - The specification
  - The references to the public items



Packages

- Package subprogram can be mapped to an external Java or C subprogram in either the package specification or package body
- The external subprogram name, parameter types, and return type are mapped to their SQL counterparts using a **call specification**
- A call specification
  - Declares a Java method or a C language subprogram so that it can be invoked from PL/SQL
  - Tells the database which Java method, or which named subprogram in which shared library, to invoke when an invocation is made
  - Tells the database what type conversions to make for the arguments and return value
  - Use the SQL CALL statement to invoke such a method or subprogram

Packages

- The AUTHID clause of the package specification determines whether the subprograms and cursors in the package run with the privileges of their definer (the default) invoker
  - Their unqualified references to schema objects are resolved in the schema of the definer or invoker

CREATE OR REPLACE PACKAGE pck_name AUTHID {CURRENT_USER | DEFINER}

  - CURRENT_USER – executes with the roles and privileges of the current user (invoker)
  - DEFINER – executes with the roles and privileges of the definer

- The ACCESSIBLE BY clause of the package specification lets developers to specify a white list of PL/SQL units that can access the package

CREATE OR REPLACE PACKAGE pck_name ACCESSIBLE BY (PROCEDURE proc_name, FUNCTION func_name, etc)



Packages

- Packages support the development and maintenance of **reliable** and **reusable code** with the following features:

1. Modularity
2. Easier Application Design
3. Hidden Implementation Details
4. Added Functionality
5. Better Performance
6. Easier to Grant Roles



Packages
Modularity

- Packages enables the encapsulation in named PL/SQL modules of logically related
  - Types
  - Variables and constants
  - Subprograms
  - Cursors
  - Exceptions

- Developers can make each package
  - Easy to understand
  - Make the interfaces between packages
    - Simple
    - Clear
    - Well defined



Packages
Easier Application Design

- When designing an application, developers need initially the interface information in the package specifications
- Developers
  - Can code and compile specifications without their bodies
  - Can compile standalone subprograms that reference the packages
  - Are not required to fully define the package bodies until the applications is ready for completion



Packages
Hidden Implementation Details

- Packages let developers
  - Share interface information in the package specification
  - Hide the implementation details in the package body
- Hiding the implementation details in the body has these advantages:
  - Implementation details can be changed without affecting the application interface
  - Application users cannot develop code that depends on implementation details that might change



Packages
Added Functionality

- Package public variables and cursors
  - **Can persist for the life of a session**
  - Can be shared by all subprograms that run in the environment
  - Permits the maintenance of data across transactions without storing them in the database



Packages
Better Performance

- Oracle Database loads the whole package into memory the first time a package subprogram is invoked
- Subsequent invocations of other subprograms in same the package require no disk I/O
- Packages prevent
  - Cascading dependencies
  - Unnecessary recompiling
- If the body of a package function is changed:
  - Oracle Database does not recompile other subprograms that invoke the function
  - Reason: these subprograms depend only on the parameters and return value that are declared in the specification



Packages
Easier to Grant Roles

- Developers and administrators can grant roles on the package, instead of granting roles on each object in the package



Package Specifications

- To create a package specification, use the CREATE PACKAGE statement

```
CREATE [OR REPLACE] PACKAGE package_name
[IS|AS]
    public declarations;
end [package_name];
```



Package Specifications

- A package specification declares public items
  - The scope of a public item is the schema of the package
  - A public item is visible everywhere in the schema
  - To reference a public item that is in scope but not visible, qualify it with the package name **pck_name.public_item_name**
  - Each public item declaration has all the information needed to use the item

- The ACCESSIBLE BY clause must be included in the package specification to restrict the use of a package to specified PL/SQL units



Package Specifications
Public Declarations

- Types, variables, constants, subprograms, cursors, and exceptions used by multiple subprograms
  - A type defined in a package specification is either a PL/SQL user-defined subtype or a PL/SQL composite type
  - A PL/SQL composite type defined in a package specification is incompatible with an identically defined local or standalone type

- Associative array types of standalone subprogram parameters
  - An associative array type cannot be declared at the schema level
  - The type of that variable must be declared in a package specification to pass an associative array variable as a parameter to a standalone subprogram
  - Doing so makes the type available to both
    - The invoked subprogram (which declares a formal parameter of that type)
    - To the invoking subprogram or anonymous block (which declares a variable of that type)



Package Specifications
Public Declarations

- Variables that must remain available between subprogram invocations in the same session
- Subprograms that read and write public/private variables
  - Write "get" and "set" subprograms (i.e., setters and getters)
  - Provide these subprograms to discourage package users from reading and writing public variables directly
- Subprograms that invoke each other
- Overloaded subprograms
  - Are variations of the same subprogram
  - Have the same name but different formal parameters (and data type for the parameters)



Package Body

- To create a package body, use the CREATE PACKAGE BODY statement

```
CREATE [OR REPLACE] PACKAGE
BODY package_name
[IS|AS]
    public definitions;
    private declarations;
    private definitions;
BEGIN
EXCEPTION
END [package_name];
```



Package Body

- A package body is required if a package specification declares cursors or subprograms (otherwise, it is optional)
- The package body and package specification must be in the same schema
- Every cursor or subprogram declaration in the package specification must have a corresponding definition in the package body
- The headings of corresponding subprogram declarations and definitions must match word for word (except for white space)
- The cursors and subprograms declared in the package specification and defined in the package body are public items that can be referenced from outside the package



Package Body

- The package body can also declare and define private items
  - That cannot be referenced from outside the package
  - Are necessary for the internal workings of the package

- The body can have an initialization part with statements initialize public variables and do other one-time setup steps
  - The initialization part runs only the first time the package is referenced
  - The initialization part can include an exception handler

- The package body can be changed without changing
  - The specification
  - The references to the public items.

Package Example

```
DROP TABLE people;

CREATE TABLE people (name, salary, deptno) AS
SELECT first_name || ' ' || last_name, salary, department_id
FROM employees;


CREATE OR REPLACE PACKAGE pkg_people IS
  CURSOR c RETURN people%ROWTYPE;
  PROCEDURE bonus;
END pkg_people;
/
```

```
BEGIN
  pkg_people.bonus;
END;
/
```



Package Example

```
CREATE OR REPLACE PACKAGE BODY pkg_people IS
  CURSOR c RETURN people%ROWTYPE IS SELECT * FROM people;

  FUNCTION cbonus(sal people.salary%type,
                  idd people.deptno%type,
                  avgsal OUT float)
  RETURN number
  IS
  BEGIN
    SELECT avg(salary) INTO avgsal
    FROM people WHERE deptno=idd;
    IF avgsal > sal THEN RETURN sal*0.2;
    ELSE RETURN sal*0.1;
    END IF;
  END cbonus;

  PROCEDURE bonus
  IS
    avgsal float(10);
    bonus float(10);
  BEGIN
    FOR p IN c LOOP
      bonus := cbonus(p.salary, p.deptno, avgsal);
      DBMS_OUTPUT.PUT_LINE(p.name||' ia un bonus de '
                           || bonus || ' (' || p.salary
                           || ' ~ ' || avgsal || ')');
    END LOOP;
    IF c%ISOPEN THEN
      CLOSE c;
    END IF;
  END bonus;
END pkg_people;
/
```



Package Example

```
CREATE OR REPLACE PACKAGE test IS
  FUNCTION power(x float) RETURN float;
  FUNCTION power(x float, y float) RETURN float;
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
  PROCEDURE power(x float, y float, z IN OUT float) IS
  BEGIN
    z := x ** y;
  END;

  FUNCTION power(x float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, 2, rez);
    RETURN rez;
  END;

  FUNCTION power(x float, y float) RETURN float IS
    rez float;
  BEGIN
    test.power(x, y, rez);
    RETURN rez;
  END;
END test;
/

BEGIN
  dbms_output.put_line(test.power(2)); -- 4
  dbms_output.put_line(test.power(2, 5)); -- 32
END;
/
```



Package Instantiation and Initialization

- Oracle Database instantiates the package for that session when a session references a package item
- Every session that references a package has its own instantiation of that package
- When Oracle Database instantiates a package, it also initializes it
- Initialization includes whichever of the following are applicable:
  - Assigning initial values to public constants
  - Assigning initial values to public variables whose declarations specify them
  - Executing the initialization part of the package body



Package State

- The values of the variables, constants, and cursors that a package declares (in either its specification or body) comprise its **package state**
- If a PL/SQL package declares at least one variable, constant, or cursor, then the package is stateful (otherwise, it is stateless)
- Each session that references a package item has its own instantiation of that package.
- If the package is stateful, the instantiation includes its state



Package State

- The package state persists for the life of a session, except in these situations:
  - The package is SERIALLY_REUSABLE
  - The package body is recompiled.
  - Any of the session's instantiated packages are invalidated and revalidated

- Oracle Database treats a package as stateless if its state is constant for the life of a session (or longer).

- This is the case for a package whose items are all compile-time constants.



Packages SERIALLY_REUSABLE

- SERIALLY_REUSABLE packages let developers design applications that manage memory better for scalability
- If the package is **NOT SERIALLY_REUSABLE**
  - The package state is stored in the user global area (UGA) for each user
  - The amount of UGA memory needed increases linearly with the number of users, limiting scalability
  - The package state can persist for the life of a session, locking UGA memory until the session ends



Packages SERIALLY_REUSABLE

- If a package is SERIALLY_REUSABLE
  - The package state is stored in the work area in a small pool in the system global area (SGA)
  - The package state persists only for the life of a server call
  - After the server call, the work area returns to the pool
  - If a subsequent server call references the package, then Oracle Database reuses an instantiation from the pool
  - By reusing an instantiation
    - The package re-initializes the state
    - The changes made to the package state in previous server calls are invisible



Packages SERIALLY_REUSABLE

- Include the SERIALLY_REUSABLE pragma in the package specification and, if it exists, the package body

```
CREATE OR REPLACE PACKAGE pkg IS
  PRAGMA SERIALLY_REUSABLE;

CREATE OR REPLACE PACKAGE BODY pkg IS
  PRAGMA SERIALLY_REUSABLE;
```



Packages SERIALLY_REUSABLE

- The work unit is a server call for a SERIALLY_REUSABLE package
- Public variables can be used only within the work unit
- After the work unit (server call) of a SERIALLY_REUSABLE package completes, Oracle Database does the following:
  - Closes any open cursors
  - Frees some non-reusable memory (for example, memory for collection and long VARCHAR2 variables)
  - Returns the package instantiation to the pool of reusable instantiations kept for this package



Packages SERIALLY_REUSABLE

- An explicit cursor in a SERIALLY_REUSABLE package
  - Remains open until either
    - It is close explicitly
    - Its work unit (server call) ends
  - A new server call must be made to re-open the cursor
  - A server call can be different from a subprogram invocation



Package Example

```
CREATE OR REPLACE PACKAGE test IS
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END test;
/

CREATE OR REPLACE PACKAGE BODY test IS
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(test.getN);
    test.setN;
    dbms_output.put_line(test.getN);
    test.setN(40);
    dbms_output.put_line(test.getN);
END;
/
```

```
CREATE OR REPLACE PACKAGE sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    FUNCTION getN RETURN NUMBER;
    PROCEDURE setN(val NUMBER := 10);
END sr_test;
/

CREATE OR REPLACE PACKAGE BODY sr_test IS
    PRAGMA SERIALLY_REUSABLE;
    n NUMBER;
    FUNCTION getN RETURN NUMBER
    IS
    BEGIN
        RETURN n;
    END getN;

    PROCEDURE setN(val NUMBER := 10)
    IS
    BEGIN
        n := val;
    END;
BEGIN
    n := 20;
END sr_test;
/

set serveroutput on
BEGIN
    dbms_output.put_line(sr_test.getN);
    sr_test.setN;
    dbms_output.put_line(sr_test.getN);
    sr_test.setN(40);
    dbms_output.put_line(sr_test.getN);
END;
/
```



Package Example

```sql
CREATE OR REPLACE PACKAGE pkg IS
  n NUMBER;
END pkg;
/

CREATE OR REPLACE PACKAGE BODY pkg IS
BEGIN
  n := 5;
END pkg;
/

CREATE OR REPLACE PACKAGE sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
  n NUMBER;
END sr_pkg;
/

CREATE OR REPLACE PACKAGE BODY sr_pkg IS
  PRAGMA SERIALLY_REUSABLE;
BEGIN
  n := 5;
END sr_pkg;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 5
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/

BEGIN
  pkg.n := 10;
  sr_pkg.n := 10;
END;
/

BEGIN
  DBMS_OUTPUT.PUT_LINE('pkg.n: ' || pkg.n); -- pkg.n: 10
  DBMS_OUTPUT.PUT_LINE('sr_pkg.n: ' || sr_pkg.n); -- sr_pkg.n: 5
END;
/
```



Package Writing Guidelines

- Become familiar with the packages that Oracle Database supplies, and avoid writing packages that duplicate their features
- Keep your packages general so that future applications can reuse them
- Design and define the package specifications before the package bodies
- In package specifications, declare only items that must be visible to invoking programs to
  - Prevent other developers from building unsafe dependencies on your implementation details
  - Reduce the need for recompilation
  - Prevent the need to recompile any subprograms that invoke the public subprograms of the package if the package specification changes
  - Remove the need for recompiling the subprograms if only the package body changes



Package Writing Guidelines

- Declare public cursors in package specifications and define them in package bodies
  - Hides cursors' queries from package users
  - Developers can change cursors' queries without changing cursor declarations

- Assign initial values in the initialization part of the package body instead of initializing them when declared
  - The code for computing the initial values can be more complex and better documented
  - If computing an initial value raises an exception, the initialization part can handle it with its own exception handler.
  - Make helper packages available only to the API if the database applications have several PL/SQL packages



Overview

- Packages
- Triggers



Triggers

- A trigger is a named PL/SQL unit that is stored in the database and can be invoked repeatedly
- A developer
  - Can enable and disable a trigger
  - Cannot explicitly invoke a trigger
- If the trigger is enabled, the database automatically invokes the trigger whenever its triggering event occurs
- If the trigger is disabled, the database does not invoke it
- To create a trigger, use the CREATE TRIGGER statement



Triggers

- At creation time, specify
  - The **triggering event in terms of triggering statements and the item on which they act**
  - The **timing point**, which determines whether the **trigger fires before or after** the triggering statement runs
  - Whether it **fires for the entire event or each row** that the triggering statement affects
- By default, a trigger is created in the enabled state
- The trigger is said to be "created on" or "defined on" the item, which is either a table, a view, a schema, or the database



Triggers

- A **DML trigger**
  - Is created on a table or view
  - The triggering event is composed of DML statements
- A **cross edition trigger**
  - Is a DML trigger for use for edition-based redefinition
- A **system trigger**
  - Is created on a schema or the database
  - The triggering event is composed of either DDL or database operation statements



Triggers

- A **conditional trigger**
  - Is a DML or system trigger
  - Has a WHEN clause that specifies a SQL condition that the database evaluates for each row that the triggering statement affects



Triggers

- An **INSTEAD OF trigger** is either:
  - A DML trigger created on either
    - A non-editioning view
    - A nested table column of a non-editioning view
  - A system trigger defined on a CREATE statement
  - The database fires the INSTEAD OF trigger instead of running the triggering statement



Triggers

- NOTE: When a trigger fires, tables that the trigger references might be undergoing changes made by SQL statements in other users' transactions
- SQL statements running in triggers follow the same rules that standalone SQL statements do
- Specifically:
  - Queries in the trigger see the current read-consistent materialized view of referenced tables and any data changed in the same transaction
  - Updates in the trigger wait for existing data locks to be released before proceeding



Triggers

- Triggers enable the customization of the database management system

- Triggers can be used to:
  - Automatically generate virtual column values
  - Log events
  - Gather statistics on table access
  - Modify table data when DML statements are issued against views
  - Enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - Publish information about
    - Database events
    - User events
    - SQL statements to subscribing applications



Triggers

- Triggers can be used to:
  - Prevent DML operations on a table after regular business hours
  - Prevent invalid transactions
  - Enforce complex business rules that cannot be defined with constraints
  - Enforce complex referential integrity rules that cannot be defined with constraints
- Caution:
  - Triggers are not reliable security mechanisms, because they are programmatic and easy to disable
  - For high-assurance security, use Oracle Database Vault



How Triggers and Constraints Differ

- Although triggers and constraints can constrain data input, they differ significantly
- A trigger always applies to new data only
- Example: Enforce NULL constraint
  - A table column does not have a null constraint
  - A trigger can prevent a DML statement from inserting NULL values
  - But the table can contain NULL values that were inserted
    - Before the trigger was defined
    - While the trigger was disabled



How Triggers and Constraints Differ

- A constraint can apply either to new data only (like a trigger) or to both new and existing data
- Constraints behavior depend on the constraint state
- Constraints are easier to write and less error-prone than triggers that enforce the same rules
- Triggers can enforce some complex business rules that constraints cannot



Triggers

- Oracle strongly recommends to use triggers to constrain data input only in these situations:
  - To enforce referential integrity when child and parent tables are on different nodes of a distributed database
  - To enforce complex business or referential integrity rules that you cannot define with constraints



DML Triggers

- A DML trigger
  - Is created on either a table or view
  - Its triggering event is composed of the DML statements DELETE, INSERT, and UPDATE
- To create a trigger that fires in response to a MERGE statement create triggers on the INSERT and UPDATE statements to which the MERGE operation decomposes
- A DML trigger is either
  - Simple
  - Compound



DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before the triggering statement runs
    - The trigger is called
      - A *BEFORE statement trigger*
      - *Statement-level BEFORE trigger*
  - After the triggering statement runs
    - The trigger is called
      - An *AFTER statement trigger*
      - *Statement-level AFTER trigger*



DML Triggers

- A **simple DML trigger** fires at exactly one of these timing points:
  - Before each row that the triggering statement affects
    - The trigger is called
      - A *BEFORE each row trigger*
      - *Row-level BEFORE trigger*
  - After each row that the triggering statement affects
    - The trigger is called
      - An *AFTER each row trigger*
      - *Row-level AFTER trigger*



DML Triggers

- A **compound DML trigger** created on a table or editioning view can fire at one, some, or all of the preceding timing points
- Compound DML triggers help define an approach where the actions implemented for the various timing points manage to share common data
- A simple or compound DML trigger that fires at row level can access the data in the row that it is processing
- An **INSTEAD OF DML trigger** is a DML trigger created on either a non-editioning view or a nested table column of a non-editioning view



DML Triggers

- Syntax:

```
CREATE [OR REPLACE] trigger_name
  {BEFORE | AFTER | INSTEAD OF}
  {INSERT | UPDATE [OF column_list] | DELETE
     [OR {INSERT | UPDATE [OF column_list] | DELETE} ...]}
  ON [table_name | view]
  [referencing clauses]
  [FOR EACH ROW]
  [WHEN (conditions)]
DECLARE
   trigger_variables
BEGIN
    statements
END;
/
```



DML Trigger Example

```
1 CREATE OR REPLACE TRIGGER t1
2   AFTER DELETE
3   ON employees
4   FOR EACH ROW
5 BEGIN
6   DBMS_OUTPUT.PUT_LINE('Deleting');
7 END;
8 /
```



DML Triggers
Conditional Predicates

- The triggering event of a DML trigger can be composed of multiple triggering statements
- When one of them fires the trigger, the trigger can determine which one by using conditional predicates
- A conditional predicate can appear wherever a BOOLEAN expression can appear

Conditional Predicate	TRUE if and only if:
INSERTING	An INSERT statement fired the trigger.
UPDATING	An UPDATE statement fired the trigger.
UPDATING ('column')	An UPDATE statement that affected the specified column fired the trigger.
DELETING	A DELETE statement fired the trigger.



DML Trigger
Conditional Predicates Example

```
 1 CREATE OR REPLACE TRIGGER t2
 2   BEFORE INSERT OR UPDATE OF salary, department_id OR DELETE
 3   ON employees
 4 BEGIN
 5   CASE
 6     WHEN INSERTING THEN
 7       DBMS_OUTPUT.PUT_LINE('Inserting');
 8     WHEN UPDATING('salary') THEN
 9       DBMS_OUTPUT.PUT_LINE('Updating salary');
10     WHEN UPDATING('department_id') THEN
11       DBMS_OUTPUT.PUT_LINE('Updating department ID');
12     WHEN DELETING THEN
13       DBMS_OUTPUT.PUT_LINE('Deleting');
14   END CASE;
15 END;
16 /
```



Correlation Names and Pseudorecords

- A trigger that fires at row level can access the data in the row that it is processing by using correlation names
- The default correlation names are OLD, NEW, and PARENT
- To change the correlation names, use the REFERENCING clause of the CREATE TRIGGER statement



Correlation Names and Pseudorecords

- If the trigger is created on a nested table
  - OLD and NEW refer to the current row of the nested table
  - PARENT refers to the current row of the parent table
- If the trigger is created on a table or view
  - OLD and NEW refer to the current row of the table or view
  - PARENT is undefined
- OLD, NEW, and PARENT are also called pseudorecords
  - They have record structure
  - Are allowed in fewer contexts than records are

Correlation Names and Pseudorecords

- The structure of a pseudorecord is table_name%ROWTYPE, where table_name is
  - The name of the table on which the trigger is created (for OLD and NEW)
  - The name of the parent table (for PARENT)

- Syntax of a pseudorecord (in the trigger body)

  :*pseudorecord_name.field_name*

- In the WHEN clause of a conditional trigger
  - A correlation name is not a placeholder for a bind variable
  - Omit the colon



Correlation Names and Pseudorecords

Triggering Statement	OLD.field Value	NEW.field Value
INSERT	NULL	Post-insert value
UPDATE	Pre-update value	Post-update value
DELETE	Pre-delete value	NULL



Correlation Names and Pseudorecords

- The restrictions on pseudorecords are:
  - A pseudorecord cannot appear in a record-level operation
    - For example, the trigger cannot include this statement: **:NEW := NULL;**
  - A pseudorecord cannot be an actual subprogram parameter
    - A pseudorecord field can be an actual subprogram parameter
  - The trigger cannot change OLD field values
    - Trying to do so raises ORA-04085
  - If the triggering statement is DELETE, then the trigger cannot change NEW field values.
    - Trying to do so raises ORA-04084
  - An AFTER trigger cannot change NEW field values, because the triggering statement runs before the trigger fires.
    - Trying to do so raises ORA-04084



Correlation Names and Pseudorecords

- A BEFORE trigger can change NEW field values before a triggering INSERT or UPDATE statement puts them in the table
- If a statement triggers both a BEFORE trigger and an AFTER trigger and the BEFORE trigger changes a NEW field value, then the AFTER trigger "sees" that change



Correlation Names and Pseudorecords

```
 1 CREATE OR REPLACE TRIGGER t3
 2   BEFORE DELETE OR INSERT OR UPDATE ON employees
 3   REFERENCING NEW AS n
 4   FOR EACH ROW
 5   WHEN (n.job_id = 'ST_CLERK' and OLD.salary < 4000 )
 6 DECLARE
 7   sal_diff  NUMBER;
 8 BEGIN
 9   sal_diff  := :n.salary  - :OLD.salary;
10   DBMS_OUTPUT.PUT(:n.last_name || ': ');
11   DBMS_OUTPUT.PUT('Old salary = ' || :OLD.salary || ', ');
12   DBMS_OUTPUT.PUT('New salary = ' || :n.salary || ', ');
13   DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
14 END;
15 /
16
17 update employees set salary = salary * 1.3;
```



INSTEAD OF DML Triggers

- An INSTEAD OF DML trigger is a DML trigger created on a non-editioning view, or on a nested table column of a non-editioning view.
- The database fires the INSTEAD OF trigger instead of running the triggering DML statement
- An INSTEAD OF trigger
  - Cannot be conditional (no WHEN clause)
  - Is the only way to update a view that is not inherently updatable
    - Design the INSTEAD OF trigger to determine what operation was intended and do the appropriate DML operations on the underlying tables
  - Is always a row-level trigger
  - Can read OLD and NEW values, but cannot change them



INSTEAD OF DML Triggers

```
CREATE OR REPLACE VIEW cr AS
  SELECT region_id, region_name, country_id, country_name
  FROM regions natural join countries;

CREATE OR REPLACE TRIGGER insert_cr INSTEAD OF INSERT ON cr
DECLARE
    ok number;
    duplicate_info EXCEPTION;
    PRAGMA EXCEPTION_INIT (duplicate_info, -00001);
BEGIN
  SELECT count(*) INTO ok FROM regions WHERE region_id = :new.region_id;
  IF ok=1 THEN
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  ELSE
    DBMS_OUTPUT.PUT_LINE('Region does not exist!');
    INSERT INTO regions VALUES(:new.region_id, :new.region_name);
    DBMS_OUTPUT.PUT_LINE('Region added!');
    IF :new.country_id IS NOT NULL THEN
      INSERT INTO countries VALUES(:new.country_id, :new.country_name, :new.region_id);
      DBMS_OUTPUT.PUT_LINE('Country added!');
    END IF;
  END IF;
EXCEPTION
  WHEN duplicate_info THEN
    RAISE_APPLICATION_ERROR(num=> -20107, msg=> 'Duplicate country');
END insert_cr;
/

INSERT INTO cr(region_id, country_id, country_name) VALUES(1, 'RO', 'Romania');

INSERT INTO cr(region_id, region_name, country_id, country_name)
VALUES(5, 'Oceania', 'AU', 'Australia');
```



INSTEAD OF DML Triggers

- An INSTEAD OF trigger with the NESTED TABLE clause
  - Fires only if the triggering statement operates on the elements of the specified nested table column of the view
  - Fires for each modified nested table element



INSTEAD OF DML Triggers

```
CREATE OR REPLACE TYPE job_line AS OBJECT(
  job_id VARCHAR2(10),
  job_title VARCHAR2(35),
  min_salary NUMBER(6),
  max_salary NUMBER(6)
);
/

CREATE OR REPLACE TYPE job_list IS TABLE OF job_line;
/

CREATE OR REPLACE VIEW dept_view AS
SELECT d.department_id, d.department_name,
  CAST (MULTISET (SELECT DISTINCT j.job_id, j.job_title, j.min_salary, j.max_salary
            FROM jobs j INNER JOIN employees e ON j.job_id = e.job_id
            WHERE e.department_id = d.department_id) AS job_list) dept_jobs
FROM departments d
WHERE EXISTS(SELECT employee_id FROM employees e WHERE e.department_id = d.department_id);

CREATE OR REPLACE TRIGGER dept_emplist_tr
  INSTEAD OF INSERT ON NESTED TABLE dept_jobs OF dept_view
  FOR EACH ROW
BEGIN
  DBMS_OUTPUT.PUT_LINE(:PARENT.department_name);
  DBMS_OUTPUT.PUT_LINE(:NEW.job_id||' '||:NEW.job_title||' '||:NEW.min_salary||' '||:NEW.max_salary);
  INSERT INTO jobs (job_id, job_title, min_salary, max_salary)
  VALUES (:NEW.job_id, :NEW.job_title, :NEW.min_salary, :NEW.max_salary);
END;
/

INSERT INTO TABLE(SELECT d.dept_jobs FROM dept_view d WHERE department_id = 10)
VALUES ('Tester', 'Super Tester', 2000, 5000);
```



Compound DML Triggers

- A compound DML trigger created on a table or editioning view can fire at multiple timing points

- Each timing point section
  - Has its own executable part
  - Has its own optional exception-handling part
  - Can access a common PL/SQL state

- The common state
  - Is established when the triggering statement starts
  - Is destroyed when the triggering statement completes, even when the triggering statement causes an error

- A compound DML trigger created on a non-editioning view is not really compound, because it has only one timing point section



Compound DML Triggers

- A compound trigger can be conditional, but not autonomous:
  - An autonomous subprogram
    - Can do SQL operations and commit or roll back those operations, without committing or rolling back the data in the main transaction
    - Uses **PRAGMA AUTONOMOUS_TRANSACTION**;

- Two common uses of compound triggers are:
  - To accumulate rows destined for a second table so that you can periodically bulk-insert them
  - To avoid the mutating-table error (ORA-04091):
    - A *mutating*-table is a table that
      - Is currently being modified by an UPDATE, DELETE, or INSERT statement
      - Might need to be updated by the effects of a declarative DELETE CASCADE referential integrity constraint



Compound DML Triggers

- A compound DML trigger created on a table or editioning view has at least one timing-point section

- If the trigger has multiple timing-point sections
  - They can be in any order
  - No timing-point section can be repeated
  - If a timing-point section is absent, then nothing happens at its timing point

Timing Point	Section
Before the triggering statement runs	BEFORE STATEMENT
After the triggering statement runs	AFTER STATEMENT
Before each row that the triggering statement affects	BEFORE EACH ROW
After each row that the triggering statement affects	AFTER EACH ROW



Compound DML Triggers

- Compound DML triggers have these restrictions:
  - OLD, NEW, and PARENT cannot appear
    - In the declarative part
    - The BEFORE STATEMENT section
    - The AFTER STATEMENT section
  - Only the BEFORE EACH ROW section can change the value of NEW
  - A timing-point section cannot handle exceptions raised in another timing-point section
  - If a timing-point section includes a GOTO statement, the target of the GOTO statement must be in the same timing-point section



Compound DML Triggers

- Syntax

```
CREATE OR REPLACE TRIGGER compound_trigger_name
FOR [INSERT|DELETE]UPDATE [OF column]
ON table COMPOUND TRIGGER
   -- Declarative Section (optional)
   -- Variables declared here have firing-statement duration.

   -- Executed before DML statement
   BEFORE STATEMENT IS
   BEGIN
     NULL;
   END BEFORE STATEMENT;

   -- Executed before each row change
   -- :NEW, :OLD are available
   BEFORE EACH ROW IS
   BEGIN
     NULL;
   END BEFORE EACH ROW;

   -- Executed aftereach row change
   -- :NEW, :OLD are available
   AFTER EACH ROW IS
   BEGIN
     NULL;
   END AFTER EACH ROW;

   -- Executed after DML statement
   AFTER STATEMENT IS
   BEGIN
     NULL;
   END AFTER STATEMENT;
END compound_trigger_name;
```



Compound DML Trigger Example

```
1 CREATE TABLE employee_salaries (
2   employee_id NUMBER NOT NULL,
3   change_date DATE   NOT NULL,
4   salary NUMBER(8,2) NOT NULL,
5   CONSTRAINT pk_employee_salaries PRIMARY KEY (employee_id, change_date),
6   CONSTRAINT fk_employee_salaries FOREIGN KEY (employee_id)
7     REFERENCES employees (employee_id) ON DELETE CASCADE
8 )
9 /
```



Compound DML Trigger Example

```
CREATE OR REPLACE TRIGGER maintain_employee_salaries
  FOR UPDATE OF salary ON employees COMPOUND TRIGGER
  -- Declarative Part:
  threshhold CONSTANT SIMPLE_INTEGER := 7;

  TYPE salaries_t IS TABLE OF employee_salaries%ROWTYPE
    INDEX BY SIMPLE_INTEGER;
  salaries  salaries_t;
  idx       SIMPLE_INTEGER := 0;

  PROCEDURE flush_array IS
    n CONSTANT SIMPLE_INTEGER := salaries.count();
  BEGIN
    FORALL j IN 1..n
      INSERT INTO employee_salaries VALUES salaries(j);
    salaries.delete();
    idx := 0;
    DBMS_OUTPUT.PUT_LINE('Flushed ' || n || ' rows');
  END flush_array;

  -- AFTER EACH ROW Section:
  AFTER EACH ROW IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER EACH ROW!');
    idx := idx + 1;
    salaries(idx).employee_id := :NEW.employee_id;
    salaries(idx).change_date := SYSTIMESTAMP;
    salaries(idx).salary := :NEW.salary;
    IF idx >= threshhold THEN
      flush_array();
    END IF;
  END AFTER EACH ROW;

  -- AFTER STATEMENT Section:
  AFTER STATEMENT IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('AFTER STATEMENT!');
    flush_array();
  END AFTER STATEMENT;
END maintain_employee_salaries;
/
```



Compound DML Trigger Example

```
1 UPDATE employees
2   SET salary = salary * 1.05
3   WHERE department_id = 50
4 /
5
6 SELECT employee_id, count(*) c
7   FROM employee_salaries
8   GROUP BY employee_id
9 /
```



System Triggers

- A system trigger
  - Is created on either
    - A schema
    - The database
  - Its triggering event is composed of either
    - DDL statements
    - Database operation statements
  - Fires at exactly one of these timing points:
    - Before the triggering statement runs
    - After the triggering statement runs
    - Instead of the triggering CREATE statement



Schema Triggers

- A **SCHEMA trigger**
  - Is created on a schema
  - Fires whenever the user who owns it
    - Is the current user
    - Initiates the triggering event

```
1 CREATE OR REPLACE TRIGGER drop_trigger
2   BEFORE DROP ON hr.SCHEMA
3   BEGIN
4     RAISE_APPLICATION_ERROR (
5       num => -20000,
6       msg => 'Cannot drop object');
7   END;
8 /
```



Database Triggers

- A **DATABASE trigger**
  - Is created on the database
  - Fires whenever any database user initiates the triggering event

```
 1 CREATE TRIGGER log_errors
 2   AFTER SERVERERROR ON DATABASE
 3   BEGIN
 4     IF (IS_SERVERERROR (1017)) THEN
 5       NULL;  -- (substitute code that processes logon error)
 6     ELSE
 7       NULL;  -- (substitute code that logs error code)
 8     END IF;
 9   END;
10 /
```



INSTEAD OF CREATE Triggers

- An **INSTEAD OF CREATE trigger**
  - Is a SCHEMA trigger
  - The triggering event is a CREATE statement
  - The database fires the trigger instead of executing its triggering statement

```
1 CREATE OR REPLACE TRIGGER t
2   INSTEAD OF CREATE ON SCHEMA
3   BEGIN
4     EXECUTE IMMEDIATE 'CREATE TABLE T (n NUMBER, m NUMBER)';
5   END;
6 /
```



Bibliography

- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 19c*, Oracle Corporation, 2022 [pdf]
- Usha Krishnamurthy et al. *Oracle® Database: SQL Language Reference 21c*, Oracle Corporation, 2022 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 19c*, Oracle Corporation, 2020 [pdf]
- Louise Morin et al. *Oracle® Database: Database PL/SQL Language Reference 21c*, Oracle Corporation, 2021 [pdf]